Machine Learning Operations (AIMLCZG523)
Assignment 1

CardioRisk

End-to-end MLOps for heart disease risk prediction

Data, experiment tracking, CI/CD, containerisation, Kubernetes deployment and monitoring

---

Student: Sevanth Gajula
BITS ID: 2025AE05634
Programme: M.Tech AI & ML (WILP), BITS Pilani
Repository: https://github.com/sevanth-gajula03/cardiorisk-mlops
Submitted: 20 October 2026

Contents

1. Introduction 3
1.1 Architecture overview 3
1.2 Setup and installation 3
2. Data acquisition and exploratory analysis 4
2.1 Acquisition 4
2.2 Cleaning 4
2.3 Key findings 5
3. Feature engineering and model development 6
3.1 Feature pipeline 6
3.2 Model selection and tuning 6
4. Experiment tracking with MLflow 7
5. Model packaging and reproducibility 8
6. CI/CD pipeline and automated testing 9
6.1 Tests 9
6.2 GitHub Actions workflow 9
7. Model serving and containerisation 10
8. Production deployment on Kubernetes 11
9. Monitoring and logging 12
10. Challenges and lessons learned 13
11. Conclusion and future work 14
Appendix A. Deliverables and access 14

1. Introduction

Heart disease is still the leading cause of death worldwide, and many of the warning signs - chest pain type, exercise ECG response, the number of blocked vessels - are captured in routine tests. The goal of this assignment was to build a classifier that estimates the risk of heart disease from those measurements, and, more importantly, to wrap it in the engineering practices that make a model trustworthy in production: reproducible data processing, tracked experiments, automated testing, a packaged model, a containerised API, a Kubernetes deployment and live monitoring.

I called the project **CardioRisk**. Everything described here runs from a clean checkout of the repository with one `requirements.txt`, and the same code path is exercised by the CI pipeline on every push.

1.1 Architecture overview

Figure 1 shows the whole system. Data is pulled from the UCI repository and validated, three model families are tuned and tracked in MLflow, and the winning pipeline is packaged into a Docker image tagged with the model version. GitHub Actions rebuilds and tests all of this on every push. The image is deployed to a Minikube cluster behind an nginx Ingress and a LoadBalancer service, and Prometheus and Grafana watch both the service and the model's behaviour.

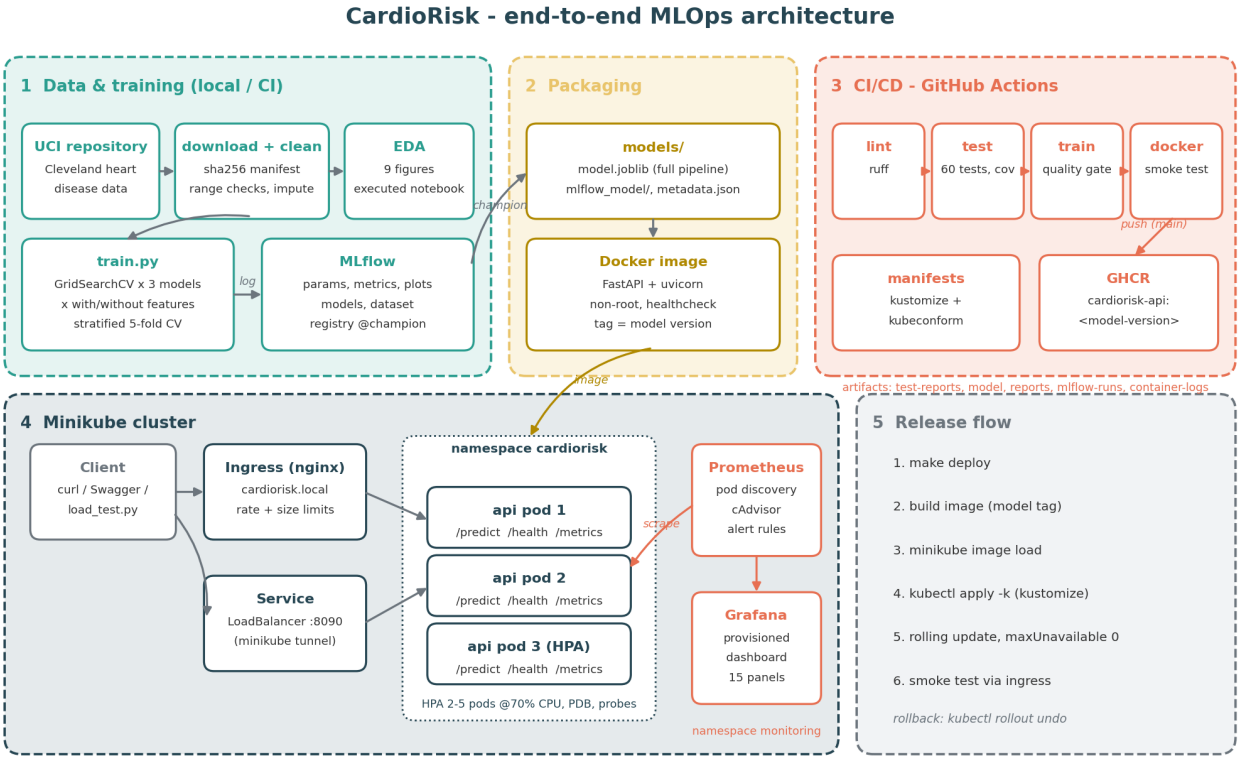


Figure 1: *End-to-end architecture of the CardioRisk system*

1.2 Setup and installation

The project targets Python 3.10+ (developed on 3.11). The full workflow is driven by a Makefile so each stage can also be run on its own:

```
git clone https://github.com/sevanth-gajula03/cardiorisk-mlops && cd cardiorisk-mlops
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt        # or: conda env create -f environment.yml
make all          # download -> clean -> EDA figures -> train -> tests
make predict      # score samples/patients.json with the packaged model
make docker-build docker-run smoke     # container on :8080 + 6 end-to-end checks
make deploy monitoring                 # Minikube deployment + Prometheus/Grafana
```

Table 1: *Technology stack*

Area	Tools
Data & modelling	pandas, NumPy, scikit-learn 1.5, matplotlib / seaborn
Experiment tracking	MLflow 3.17 (SQLite backend, model registry)
Serving	FastAPI, Pydantic, uvicorn, prometheus-client
Testing & CI	pytest + coverage, ruff, GitHub Actions, kubeconform
Containers & deployment	Docker (Colima), Minikube, kustomize, nginx Ingress, HPA
Monitoring	Prometheus 3.5, Grafana 12.1, structured JSON logs

2. Data acquisition and exploratory analysis

2.1 Acquisition

`heartrisk.data.download` downloads the official UCI archive, extracts `processed.cleveland.data` (303 patients, 13 features plus the diagnosis) and falls back to the direct file link if the archive endpoint is unavailable. It writes a `MANIFEST.json` with the source URL, timestamp and SHA-256 checksum; the same checksum is stored in the model metadata, so every trained model can be traced back to the exact bytes it was trained on. Because my network re-signs HTTPS traffic, TLS is verified against the operating system's certificate store (via `truststore`) rather than Python's bundled CA list.

2.2 Cleaning

Table 2: *Cleaning steps (logged to reports/data_quality.json)*

Step	What I did	Effect on the data
Missing values	`?` parsed as NaN; mode imputation for the coded columns	`ca`: 4 rows, `thal`: 2 rows filled
Duplicates	Exact duplicate rows dropped	None in the Cleveland file
Range checks	Values outside clinical limits in `params.yaml` treated as missing	No violations (kept as a guard for new data)
Target	Diagnosis 0-4 collapsed to 0 = no disease, 1 = disease	164 vs 139 patients (54% / 46%)
Encoding	One-hot `cp`, `restecg`, `slope`, `thal` with readable names	`heart_encoded.csv`, 23 columns

I chose the **mode rather than the median** for the two incomplete columns because they are categorical codes: a median of `thal` values 3 and 7 would be 5, which is not a valid code. Scaling is deliberately not done at this stage; it lives inside the model pipeline so it is only ever fitted on training folds.

2.3 Key findings

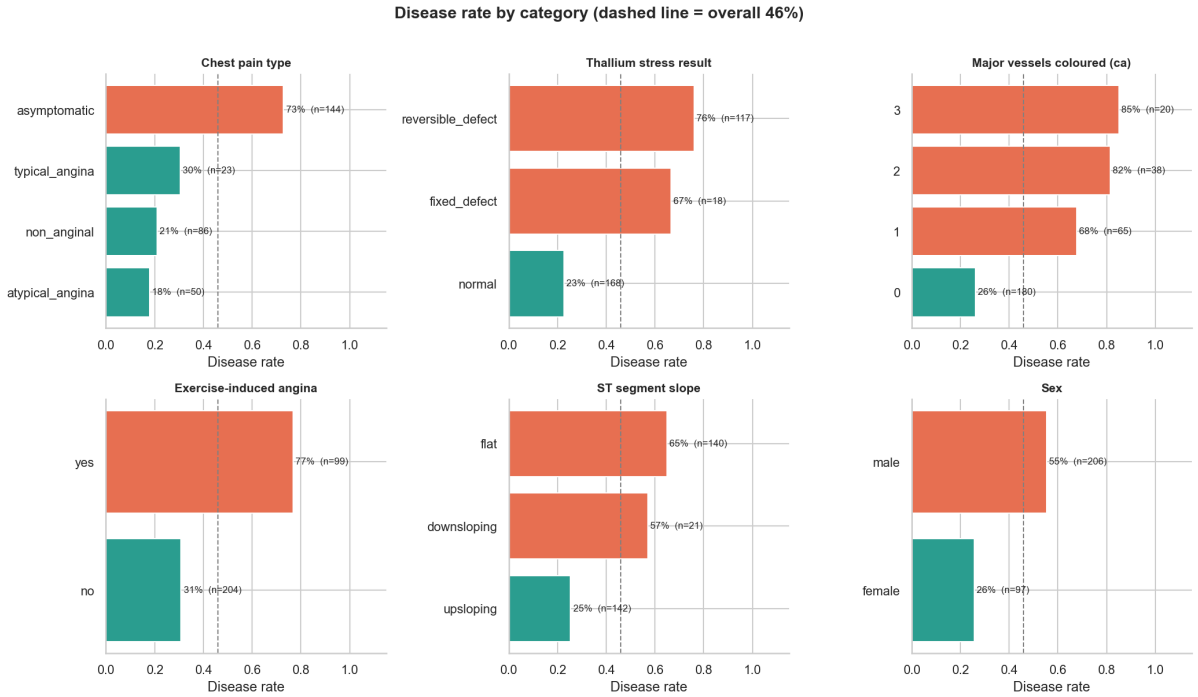


Figure 2: *Disease rate per category with group sizes; the dashed line is the overall 46% rate*

- The classes are reasonably balanced (54/46), so accuracy is meaningful, but I still report precision, recall and ROC-AUC and use stratified folds.
- The strongest signals are `thal` (reversible defect: 76% disease), `ca` (any coloured vessel: 68-85%), exercise-induced angina (77% vs 31%) and **asymptomatic chest pain (73%)** - counter-intuitive, but typical of this cohort where silent ischaemia was referred for testing.
- `chol` (r = 0.09) and `fbs` (r = 0.03) carry almost no linear signal; `oldpeak` is heavily right-skewed.
- Max heart rate falls with age for everyone, but patients with disease sit further below the `220 - age` reference, and risk rises with age much faster for men (Figure 3).

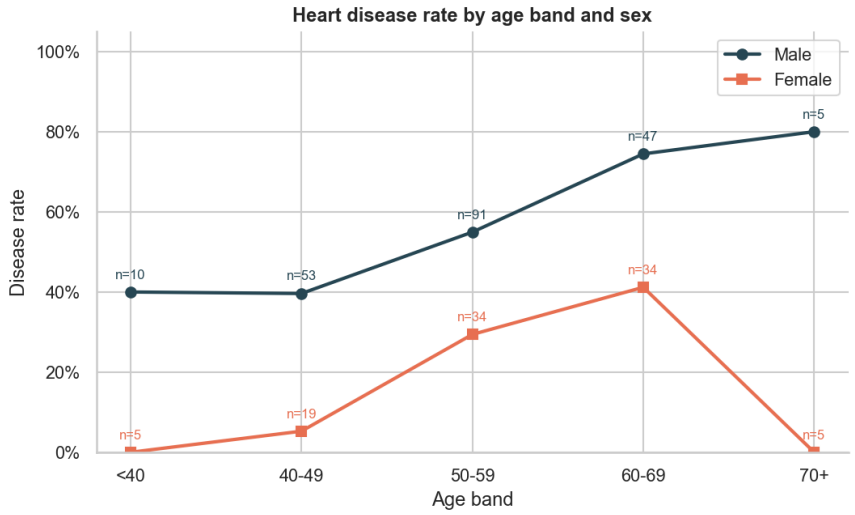


Figure 3: *Disease rate by age band and sex - the motivation for an age x sex feature*

These last two observations directly shaped the feature engineering in the next section. The executed notebook `notebooks/01_eda.ipynb` contains all nine EDA figures with commentary.

3. Feature engineering and model development

3.1 Feature pipeline

All preprocessing is one scikit-learn `Pipeline` (`src/heartrisk/features.py`), so the same object is used for training, batch scoring and the API, and nothing has to be re-implemented at serving time:

- **ClinicalFeatures** (custom transformer) adds `hr_reserve = thalach / (220 - age)` - achieved heart rate as a fraction of the age-predicted maximum - and `age_x_male = age x sex`.
- Median imputation and `StandardScaler` for continuous columns and the ordinal `ca`; a **Yeo-Johnson** power transform for the skewed `oldpeak`.
- Mode imputation and one-hot encoding (unknown categories ignored) for `cp`, `restecg`, `slope`, `thal`.

3.2 Model selection and tuning

I tuned three model families with `GridSearchCV` using stratified 5-fold cross-validation, ROC-AUC as the refit metric, and accuracy, precision, recall and F1 recorded alongside. As a personal experiment, **every model was trained twice - with and without the engineered features** - to measure whether they actually help. The data was split 80/20 (stratified, seed 42); tuning and model selection used only the training split, and the 20% hold-out was reported but never used to choose anything.

Table 3: *Results of the tuning session (5-fold CV on 242 rows, test on 61 rows)*

Run	Grid	CV ROC-AUC	Test ROC-AUC	Test recall	Train-CV gap
logreg-eng (champion)	24	**0.915 +/- 0.013**	0.948	0.929	0.010
logreg-base	24	0.908 +/- 0.014	0.953	0.929	0.025
rf-eng	36	0.903 +/- 0.021	0.945	0.893	0.067
rf-base	36	0.900 +/- 0.026	0.950	0.929	0.064
gbm-eng	16	0.894 +/- 0.024	0.956	0.929	0.073
gbm-base	16	0.883 +/- 0.028	0.960	0.929	0.078

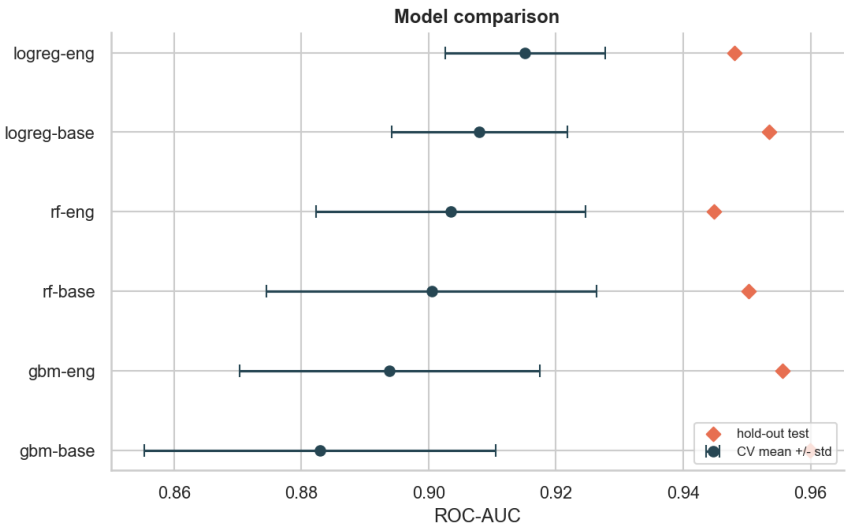


Figure 4: *CV ROC-AUC (mean +/- std) against hold-out ROC-AUC for all six runs*

Why logistic regression (C = 0.1, L2)? It has the highest and most stable cross-validated ROC-AUC, the smallest gap between training and validation scores (0.01 against roughly 0.07 for the tree ensembles, which overfit 303 rows noticeably), reasonably calibrated probabilities and readable coefficients - important for a clinical risk score. Gradient boosting looks best on the 61-row test set but worst in CV; with so few test rows that difference is within

noise, so I trusted CV. The engineered features improved CV ROC-AUC for **all three** families (+0.007, +0.003 and +0.011).

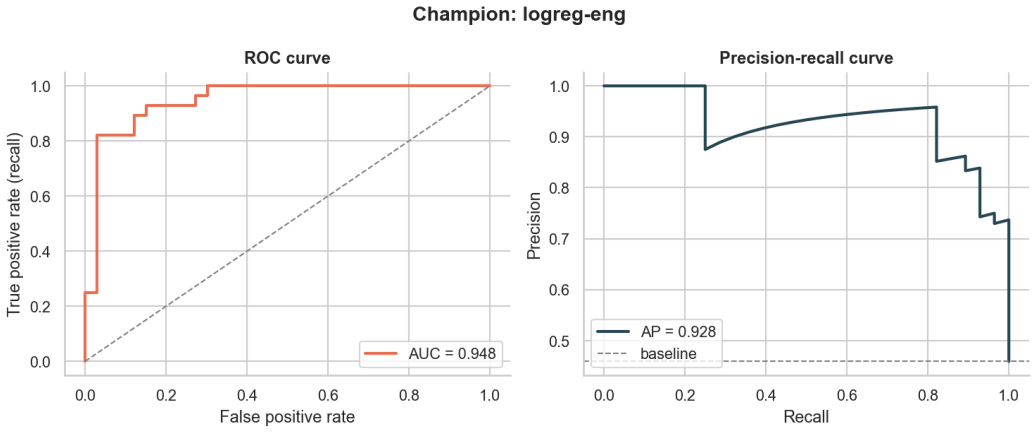


Figure 5: *Champion ROC and precision-recall curves on the hold-out set*

On the hold-out set the champion reaches accuracy 0.869, precision 0.813, **recall 0.929** and ROC-AUC 0.948. For a screening tool, high recall (few missed patients) is the more important property. The chosen configuration is finally refitted on all 303 rows for the shipped model.

4. Experiment tracking with MLflow

Each run of `python -m heartrisk.models.train` creates one parent MLflow run (a training session) with a nested child run per model and feature set. The tracking store is a local SQLite database; setting `MLFLOW_TRACKING_URI` points the same code at a remote server.

Table 4: *What is tracked for every run*

What	Logged
Parameters	model type, feature set, best hyper-parameters, grid size, CV folds, seed, data SHA-256, train/test sizes
Metrics	CV mean and std for accuracy, precision, recall, F1, ROC-AUC; train ROC-AUC; overfit gap; all test metrics
Plots	confusion matrix, ROC + PR curves, calibration curve, feature importance (each run); comparison and ROC overlay (session)
Artifacts	`cv_results.csv`, `feature_importance.csv`, `search_space.json`, `params.yaml`, model comparison table
Dataset	`heart_clean` logged as the training input with its digest
Models	every tuned pipeline with signature and input example; champion registered as `cardiorisk-classifier@champion`

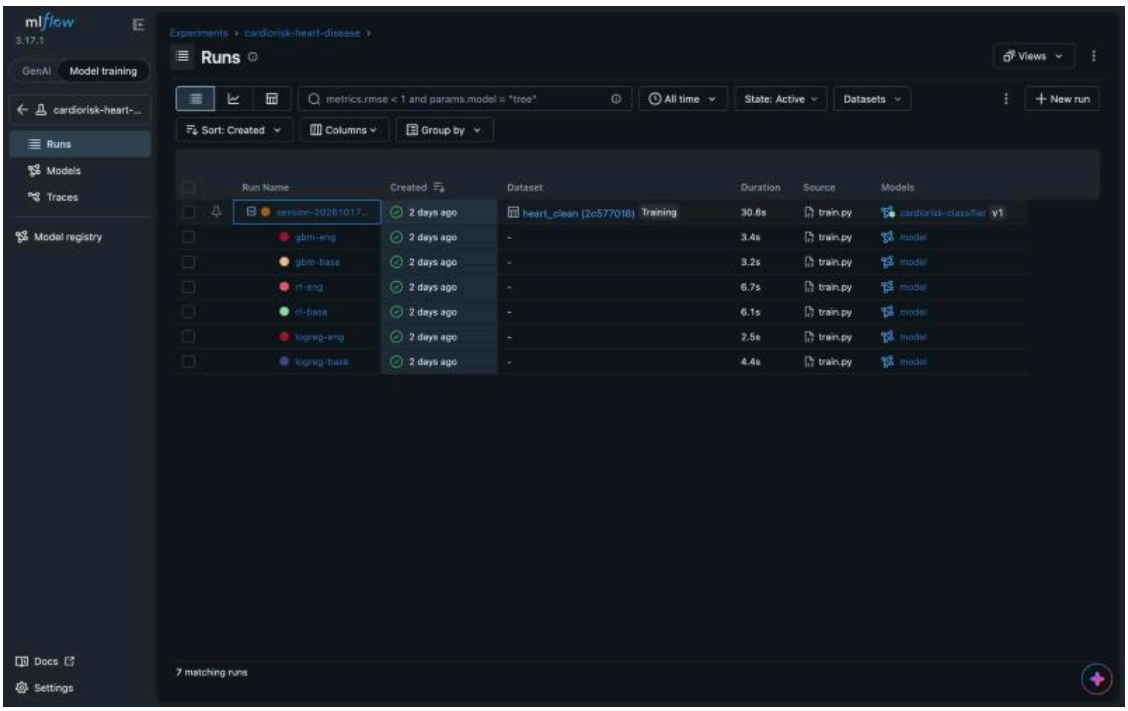


Figure 6: *MLflow: a training session with its six nested runs and the registered model*

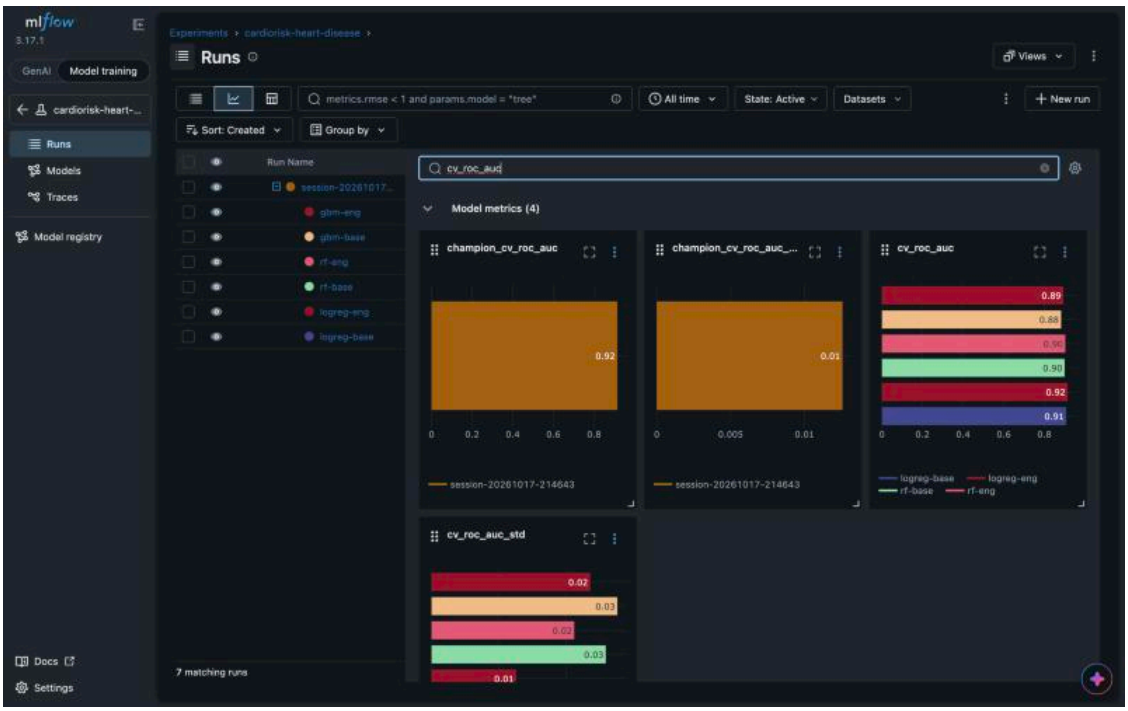


Figure 7: *MLflow chart view comparing cv_roc_auc and its standard deviation across runs*

5. Model packaging and reproducibility

The champion is written to `models/` in two equivalent formats plus metadata:

- `model.joblib` - the complete pipeline (features, preprocessing and model), used by the API.
- `mlflow_model/` - the same pipeline in MLflow format with `MLmodel`, `conda.yaml`, `requirements.txt` and an input example, loadable with `mlflow.pyfunc.load_model`.
- `metadata.json` - model version, hyper-parameters, metrics, feature list, decision threshold, training-data SHA-256 and the MLflow run IDs.

Reproducibility comes from: pinned versions in `requirements.txt` (with an `environment.yml` alternative and a slimmer `requirements-serve.txt` for the container), a single random seed and all paths and choices in `config/params.yaml`, a deterministic split, data checksums, and the preprocessing living inside the saved model. I verified this by rebuilding a fresh virtual environment from the requirements file and re-running the whole pipeline: it produced identical metrics.

MLflow 3 serialises scikit-learn models with `skops` and refuses unknown classes. Rather than hard-coding a list, the training script asks skops which types the pipeline contains (our own `ClinicalFeatures`, NumPy dtypes) and passes exactly those as trusted.

6. CI/CD pipeline and automated testing

6.1 Tests

There are **60 pytest tests (73% line coverage)** across data cleaning (missing values, duplicates, range checks, encoding), the engineered features and preprocessing pipeline, model helpers and the inference wrapper, evaluation plots, the downloader (network mocked, including the fallback path) and the API (validation limits, batch limits, request IDs, metrics). Model tests use a small synthetic dataset so they need no download.

6.2 GitHub Actions workflow

`.github/workflows/ci.yml` runs on every push to `main`, on pull requests and on demand. Jobs depend on each other, so a failure stops everything after it.

Table 5: *CI jobs*

Job	Steps	Artifacts / output
Lint	ruff (pycodestyle, pyflakes, isort, bugbear, pyupgrade, simplify) with inline PR annotations	-
Validate manifests	kustomize render of the app and monitoring stacks, strict kubeconform validation	-
Unit tests	pytest with coverage	JUnit XML, coverage XML/HTML; coverage in job summary
Train and evaluate	download, clean, EDA, full tuning session with MLflow, **model quality gate**, sample predictions	model, reports, mlflow-runs; results table in summary
Container	build image with the model trained in this run, run it, smoke test; push to GHCR on main	container logs; image size in summary

The **model quality gate** (`scripts/check_model_quality.py`) is my addition: the run fails if a retrained champion drops below CV ROC-AUC 0.85, test ROC-AUC 0.85 or test recall 0.80, so a data or code change cannot silently ship a worse model. The first run on GitHub completed all five jobs in 6 min 38 s and pushed the image to `ghcr.io/sevanth-gajula03/cardiorisk-api`.

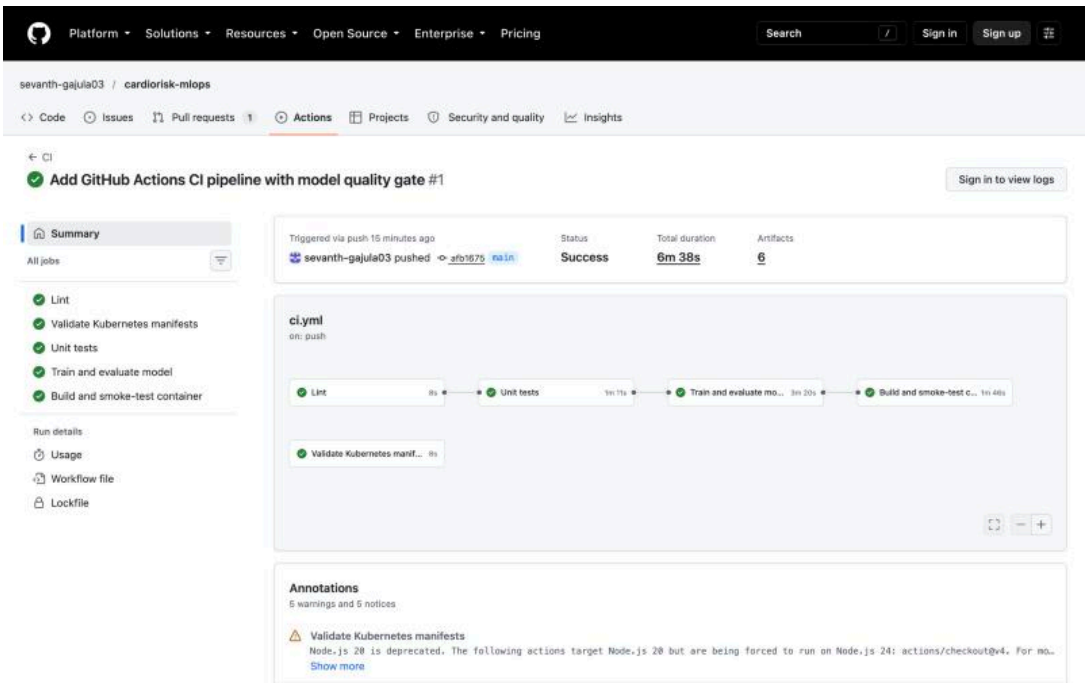


Figure 8: *Successful pipeline run: lint, manifest validation, tests, training and container jobs*

To prove the pipeline fails on errors, I opened a pull request that widens the low-risk band from 0.30 to 0.35 without updating the tests. The run stopped at the unit-test job with `AssertionError: assert 'low' == 'moderate'` in `test_risk_band[0.3-moderate]`, and training and the container build were skipped (Figure 9).

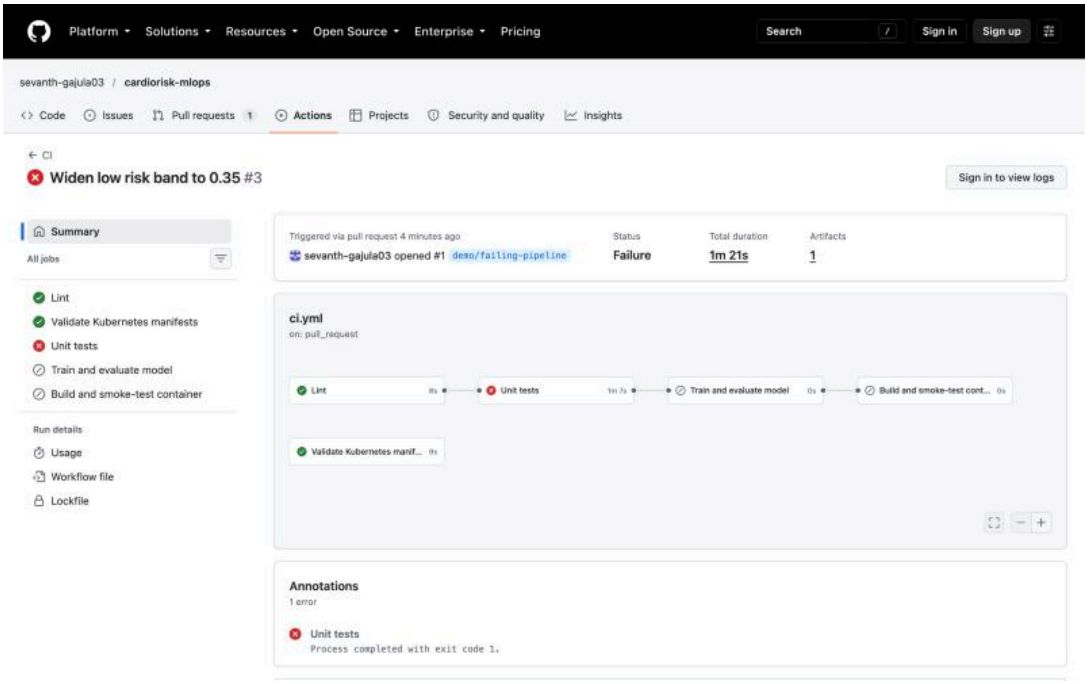


Figure 9: *The same pipeline failing on a pull request with a broken test*

7. Model serving and containerisation

The API (`src/heartrisk/api`) is built with FastAPI and exposes:

Table 6: *API endpoints*

Method	Path	Purpose
POST	/predict	score one patient; returns prediction, label, probability, confidence, risk band and top factors

Method	Path	Purpose
POST	/predict/batch	score 1-100 patients in one call
GET	/health, /ready	liveness and readiness (503 until the model is loaded)
GET	/model/info	version, hyper-parameters and metrics of the loaded model
GET	/metrics	Prometheus metrics

Every input field has clinical limits and allowed codes (Pydantic), and unknown fields are rejected, so bad input returns a clear 422 instead of a silent wrong answer. Two personal touches: each response includes a **risk band** (low < 0.3 <= moderate < 0.6 <= high) and the **three factors that pushed this patient's prediction most** (coefficient x standardised value), and every response carries the model version and a request ID that also appears in the logs.

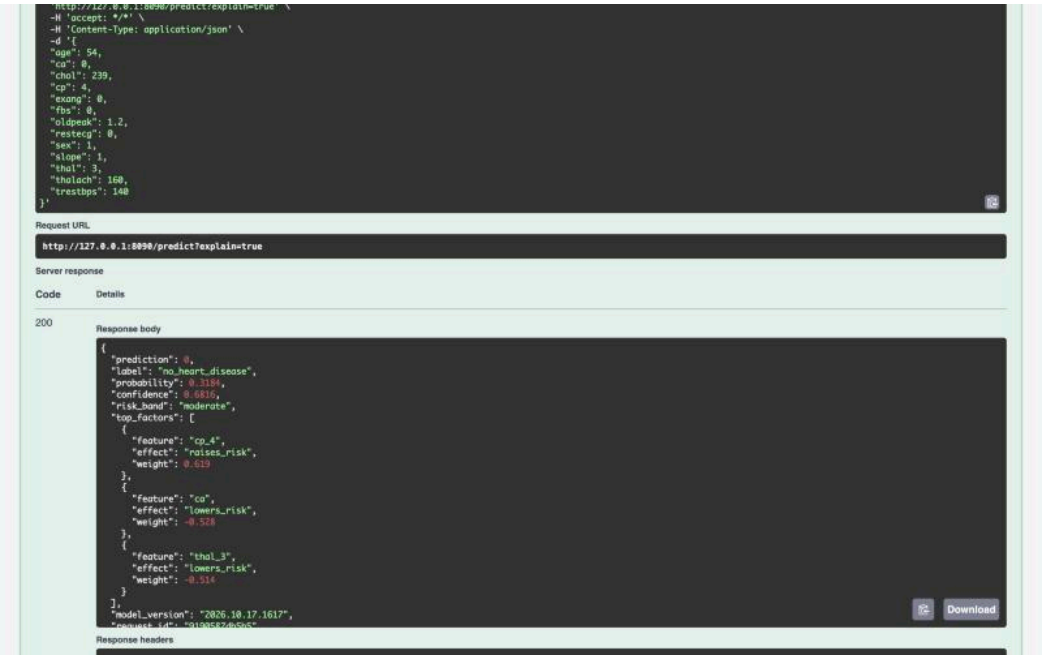


Figure 10: *A live /predict call from Swagger UI against the Kubernetes deployment*

The Dockerfile is a two-stage build: wheels are compiled in a builder stage and bind-mounted into the runtime stage so they never end up in an image layer. The runtime image runs as a non-root user (UID 10001), has a `HEALTHCHECK` on `/health`, and stores the model version and git SHA as labels. I run a single uvicorn worker per container because Prometheus counters are per-process; scaling is done with Kubernetes replicas instead. `scripts/smoke_test.sh` checks health, readiness, a known high-risk prediction, batch scoring, input validation and the metrics endpoint, and is used locally, in CI and after each deployment.

8. Production deployment on Kubernetes

The service runs on Minikube (Docker driver on Colima). Manifests live in `k8s/base` with a `k8s/overlays/minikube` kustomize overlay that pins the image tag to the model version.

Table 7: *Kubernetes resources*

Resource	Configuration
Deployment	2 replicas; rolling update with maxUnavailable 0; startup, liveness (/health) and readiness (/ready) probes; CPU/memory requests and limits; non-root, read-only root filesystem, all capabilities dropped; Prometheus annotations
Service	type LoadBalancer on port 8090 (external IP from `minikube tunnel`)
Ingress	nginx, host `cardiorisk.local`, request size and rate limits
HPA / PDB	2-5 pods at 70% CPU; at least one pod always available

`make deploy` builds the image tagged with the model version, loads it into Minikube, applies the manifests, records a change-cause, waits for the rollout and runs the smoke test through the Ingress. Retraining therefore always produces a new tag and a zero-downtime rolling update, and `kubectl rollout undo` returns to the previous model. During a load test the autoscaler scaled the deployment from two to three pods.

```
$ kubectl -n cardiorisk get deploy,svc,ingress,hpa
deployment.apps/cardiorisk-api   2/2   2   2
service/cardiorisk-api   LoadBalancer   10.107.235.53   127.0.0.1   8090:30570/TCP
ingress.networking.k8s.io/cardiorisk-api   nginx   cardiorisk.local   192.168.49.2   80
horizontalpodautoscaler.autoscaling/cardiorisk-api   cpu: 3%/70%   2   5   2

$ scripts/smoke_test.sh http://127.0.0.1:8090
  [ok]   GET /health            [ok]   GET /ready
  [ok]   high-risk patient predicted as disease
  [ok]   POST /predict/batch    [ok]   invalid input rejected    [ok]   metrics exposed
All 6 checks passed.
```

Access: `minikube tunnel` then `http://127.0.0.1:8090/docs`, or `kubectl -n ingress-nginx port-forward svc/ingress-nginx-controller 8081:80` and requests with the `Host: cardiorisk.local` header.

9. Monitoring and logging

Every request produces one JSON log line with request ID, method, route, status, latency and client, and every prediction adds a line with label, probability, risk band and model version. The request ID is taken from the caller or the Ingress `X-Request-ID` header, so one ID can be followed from nginx to the pod log to the response. The API exposes request counts and latency histograms by route and status, in-flight requests, predictions by label and risk band, the predicted-probability distribution, model-only inference time and a model-info gauge. Unknown URLs share a single `unmatched` label so scanners cannot create unbounded metric series.

I wrote a small Prometheus and Grafana stack instead of installing a large chart. Prometheus discovers API pods through the Kubernetes API using their annotations and scrapes container CPU and memory from cAdvisor. Grafana is provisioned entirely from code: the data source, the dashboard provider and a 15-panel dashboard generated by `scripts/build_dashboard.py`.

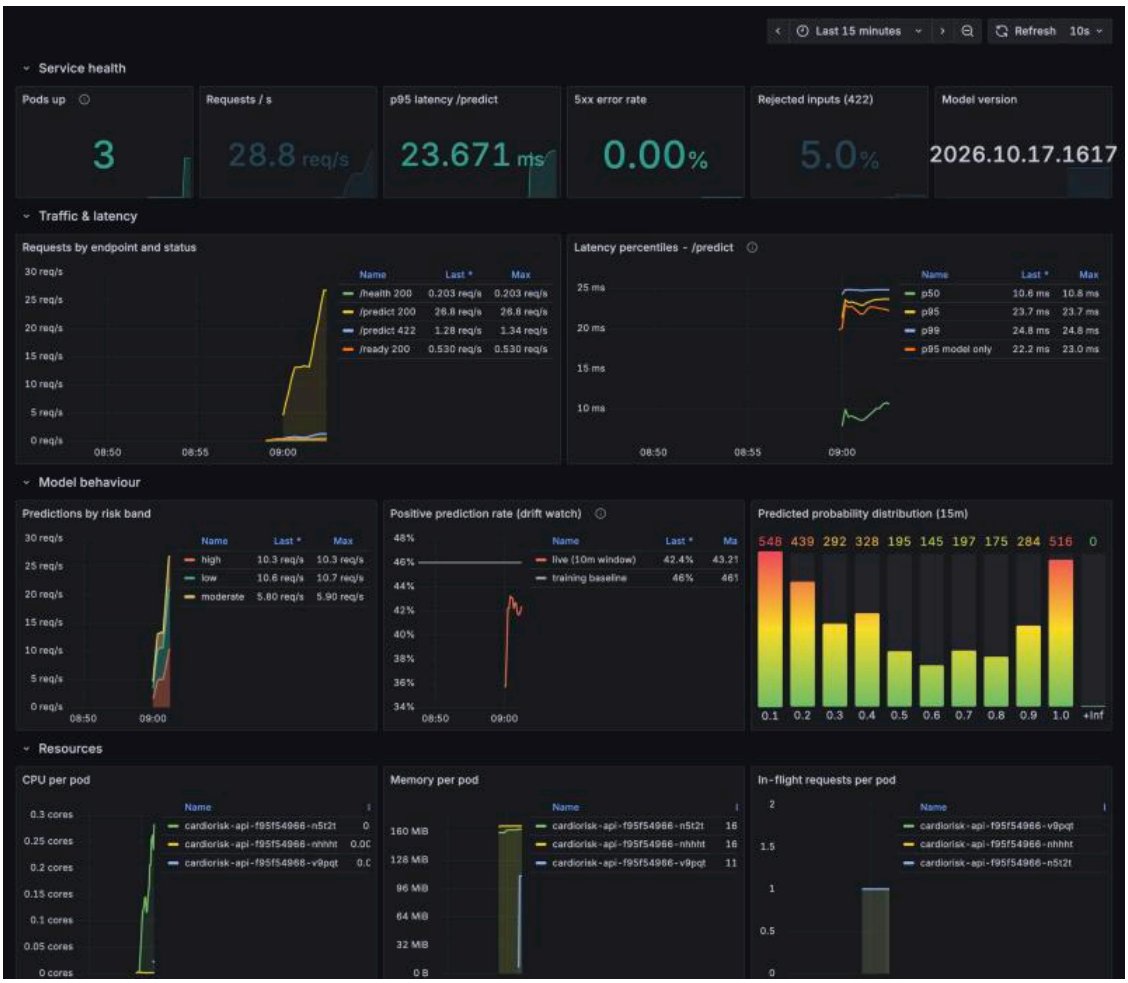


Figure 11: *Grafana dashboard under load: service health, latency, model behaviour and resources*

Alert rules cover the API being down, more than 2% server errors, p95 latency above 250 ms and more than 20% invalid requests. I also added a simple **prediction-drift** alert: the share of positive predictions over ten minutes is recorded as a series and compared with the 46% seen in training; leaving the 15-75% band raises an alert. Under load the live positive rate was about 43%, p95 latency about 24 ms and the server error rate 0%.

10. Challenges and lessons learned

- **TLS-inspecting corporate proxy.** Downloads, Minikube image pulls and Docker pulls failed with certificate errors. I used the OS trust store in Python, installed the company root and issuing CAs into Minikube (`~/.minikube/certs`, `--embed-certs`) and the Colima VM, and loaded images that still failed to pull with `minikube image load`.
- **runAsNonRoot needs a numeric user.** Pods were rejected until the image used a fixed UID (10001) that Kubernetes can verify.
- **Per-process metrics.** Two uvicorn workers made counters jump between scrapes; one worker per pod with Kubernetes replicas fixed it.
- **Small data, noisy test set.** With 61 test rows, test-set rankings flipped between runs; selecting on cross-validation and logging the train-CV gap gave a far more stable choice.
- **Testing the pipeline itself.** Running the CI steps from a fresh copy before pushing caught two regressions introduced by lint fixes before GitHub ever saw them.

11. Conclusion and future work

CardioRisk takes a small public dataset through a complete, automated MLOps lifecycle: a validated and checksummed dataset, documented EDA, tuned and tracked models, a packaged pipeline, a tested and containerised API, a Kubernetes deployment with autoscaling and zero-downtime updates, and monitoring of both the service and the model. Natural next steps are a remote MLflow server with artifact storage, promoting models through registry aliases from CI, data-drift checks on the input features (not just the predictions), probability calibration, and deploying the same manifests to a managed cluster with GitOps.

Appendix A. Deliverables and access

Table 8: *Where each deliverable lives*

Deliverable	Location
Code repository	https://github.com/sevanth-gajula03/cardiorisk-mlops
Dataset + download script	`data/`, `src/heartrisk/data/download.py`, `data/raw/MANIFEST.json`
EDA notebook / scripts	`notebooks/01_eda.ipynb`, `src/heartrisk/eda/plots.py`
Training / inference	`src/heartrisk/models/train.py`, `src/heartrisk/models/predict.py`
Tests	`tests/` (60 tests)
CI workflow	`.github/workflows/ci.yml`
Dockerfile, requirements	`Dockerfile`, `requirements.txt`, `requirements-serve.txt`, `environment.yml`
Deployment manifests	`k8s/base`, `k8s/overlays/minikube`, `k8s/monitoring`
Screenshots	`screenshots/`
Container image	`ghcr.io/sevanth-gajula03/cardiorisk-api`

Local access: after `make deploy`, run `minikube tunnel` and open `http://127.0.0.1:8090/docs`; Grafana with `make grafana` (http://localhost:3000) and Prometheus with `make prometheus` (http://localhost:9090).